

Privately Accountable Multi-Signature for Issuer-Hiding Credentials

Haochen Yang¹, Graduate Student Member, IEEE, Yong Yu², Fellow, IEEE, Qian Zhang, Keyan Xin, Huilin Li, and Dongxiao Yu³, Senior Member, IEEE

I. INTRODUCTION

Abstract—The introduction of issuer hiding further strengthens the privacy guarantees of Attribute-based Credentials (ABC) by eliminating a frequently overlooked yet serious privacy risk: the mere knowledge of an issuer's identity can reveal sensitive information of users. However, existing issuer-hiding credential systems lack effective accountability mechanisms, which hinders the tracing of malicious issuers. Furthermore, the lack of use control over credentials enables users to misuse them on unauthorized platforms. In addition, verification of a multi-issuer credential typically requires maintaining the public keys of all participating issuers, increasing storage overhead. To address these challenges, this paper proposes a Privately Accountable and Privately Verifiable Multi-Issuer Credential scheme (PAPV-MIC). The scheme inherently provides issuer-hiding guarantees with constant-size credentials, while still allows an authorized entity holding a tracing secret key to reveal the identities of malicious issuers, thereby achieving private accountability. For verification, the public keys of all issuers trusted by the verifier are compressed into a single group element, which the verifier combines with its own secret key to complete verification without storing the public keys of all issuers. This design guarantees private verifiability, prevents credential misuse, and significantly reduces storage overhead. PAPV-MIC is built upon a novel primitive named Privately Accountable and Privately Verifiable Multi-Signature (PAPVMS). We formally prove the private accountability, private verifiability, and unforgeability of both PAPVMS and PAPV-MIC in the random oracle model, and demonstrate their practical efficiency through comprehensive analysis and experimental evaluation.

Index Terms—Attribute-based credential, issuer-hiding, multi-signature, privacy-preserving.

WEB 3.0¹ promotes a decentralized paradigm centered on user data sovereignty by reducing reliance on centralized authorities. Within this paradigm, self-sovereign identity emerges as a key infrastructure for enabling secure identity management and authentication under minimal trust assumptions. ABC, typically realized via digital signatures, allow users to privately prove possession of certified attributes while minimizing information disclosure. As a result, ABC play a central role in enabling privacy-preserving authentication in decentralized environments.

With recent advances in the ABC field, numerous schemes are proposed in both academia and industry to support diverse functionalities and application scenarios, including selective disclosure credentials [1], federated authentication credentials [2], and delegatable credentials [3]. In decentralized environments, joint issuance of credentials becomes a common paradigm. In decentralized governance frameworks, such as Decentralized Autonomous Organizations (DAOs), often require multiple entities to jointly authorize the issuance of permissions or qualifications, thereby ensuring decentralized authorization [4]. In multi-issuer settings, issuer identities may inadvertently reveal auxiliary contextual information beyond the credential content itself, since issuer-related metadata (e.g., geographic location) can be implicitly inferred from them. To mitigate privacy leakage arising from issuer identity disclosure, issuer-hiding multi-issuer credential systems attract increasing research attention [5].

However, in multi-issuer credential systems, a fundamental challenge arises in balancing issuer privacy and accountability. Hidden issuers without effective accountability mechanisms may exploit their anonymity to issue illegitimate credentials, leading to security and trust risks. A recent fraud case reveals that two nursing schools in Florida issue credentials that are formally verifiable but fraudulent, enabling unqualified individuals to take the U.S. national nursing licensure examinations, which results in an estimated loss of \$6.94 million.² Moreover, the inherent replicability of ABCs complicates usage control, as credentials can be copied or reused without authorization. In multi-issuer setting, the verification of credentials typically requires storing public keys of all issuers which introduces substantial storage

Received 20 January 2026; revised 28 April 2026; accepted 11 June 2026. Date of publication 16 June 2026; date of current version 1 September 2026. This work was supported in part by the National Cryptologic Science Fund of China under Grant 2025NCSF02025, in part by the National Natural Science Foundation of China under Grant U24B20149, Grant 62272385, and Grant U23A20302, in part by the Key Research and Development Program of Shaanxi under Grant 2024GX-ZDCYL-01-09, in part by the Major Program of Shandong Provincial Natural Science Foundation for the Fundamental Research under Grant ZR2022ZD03, and in part by the Fundamental Research Funds for the Central Universities under Grant GK202603010. (Corresponding author: Yong Yu.)

Haochen Yang, Yong Yu, and Keyan Xin are with the School of Artificial Intelligence and Computer Science, Shaanxi Normal University, Xi'an 710119, China (e-mail: haochen-yang@foxmail.com; yuyong@snnu.edu.cn; keyanxin@snnu.edu.cn).

Qian Zhang is with the School of Cyber Engineering, Xidian University, Xi'an 710071, China (e-mail: zhang_qian@stu.xidian.edu.cn).

Huilin Li and Dongxiao Yu are with the School of Artificial Intelligence, Shandong University, Jinan 250100, China (e-mail: lihuilinlin@163.com; dxyu@sdu.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3704353

¹<https://aws.amazon.com/cn/what-is/web3/>

²<https://www.justice.gov/usao-sdfl/pr/three-defendants-sentenced-fraudulent-nursing-diploma-scheme>

overhead. This issue becomes particularly significant when frequent key updates are required [6].

To address the aforementioned issues, this paper proposes a Privately Accountable and Privately Verifiable Multi-Issuer Credential scheme (PAPV-MIC). PAPV-MIC produces credentials of constant size while simultaneously ensuring private accountability for issuers. During verification, verifiers learn only that the issuers belong to a predefined authorized set, whereas the concrete identities of the issuers remain hidden. When accountability is required, an authorized entity holding a tracing secret key can reveal the identities of the participating issuers, thereby balancing issuer privacy and accountability. Moreover, PAPV-MIC achieves private verifiability without requiring aggregation of participating issuers' public keys. A verifier performs verification using its secret key together with a constant-size verification key derived from its trusted issuer set. This design prevents credential reuse on unauthorized platforms and eliminates the need of storing public keys of all issuers, making PAPV-MIC suitable for blockchain-based systems.

Contributions: Specifically, the main contributions of this paper are summarized as follows.

- We propose PAPVMS, a novel multi-signature protocol that simultaneously achieves private accountability for signers and private verifiability for verifiers, without requiring the aggregation or storage of signers' public keys. PAPVMS allows signatures generated by multiple independent signers to be aggregated into a constant-size signature. The protocol hides the identities of participating signers, while an authorized entity holding a tracing secret key can reveal the signers when accountability is required. Verification is restricted to a designated verifier using its secret key together with a constant-size key consisting of a single group element, which prevents misuse over unauthorized platforms and eliminates the need to store signers' public keys. We prove that PAPVMS satisfies private accountability, private verifiability, and unforgeability in the random oracle model.
- We propose PAPV-MIC, a multi-issuer credential scheme built upon PAPVMS. Our scheme achieves private accountability by allowing an authorized entity holding a tracing secret key to reveal the identities of malicious issuers among the hidden issuers. Moreover, PAPV-MIC achieves private verifiability without storing the public keys of all issuers. A designated verifier validates constant-size credentials using its secret key together with a compact key derived from its trusted issuer set. We prove that PAPV-MIC satisfies private accountability, private verifiability, and credential unforgeability in the random oracle model.
- We evaluate the performance and functionality of PAPVMS and PAPV-MIC. PAPVMS is compared with existing multi-signature protocols in terms of computational complexity, storage overhead, and functionality. PAPV-MIC is evaluated experimentally for efficiency and practicality in multi-issuer settings. Results show that both PAPVMS and PAPV-MIC meet the functional and efficiency

requirements, with experiments confirming PAPV-MIC's deployability and scalability.

Organization: The remainder of this paper is organized as follows. Section II reviews the related works and identifies existing research gaps. Section III presents the preliminaries. Section IV presents the definition and construction of PAPVMS, together with its security analysis and proof. Section V presents the definition and construction of PAPV-MIC built on PAPVMS, together with its security proof and experimental evaluation. Section VI concludes the paper.

II. RELATED WORKS

In this section, we analyze existing research on attribute-based credentials as well as recent studies on issuer-hiding credentials, highlighting the advantages and limitations of current methods.

A. Attribute-Based Credentials

Attribute-based credentials enable holders to prove their attributes in a privacy-preserving manner. With the continued development of decentralized applications, traditional attribute-based credential systems relying on a single issuing authority may face limitations in certain decentralized scenarios. To better accommodate highly decentralized environments, multi-issuer credential issuance models are proposed and gradually attract increasing research attention.

Recently, Mir et al. [7] proposed an anonymous credential system whose credentials support threshold issuance, where multiple issuers jointly issue a credential in a t-out-of-n manner, thereby eliminating the reliance on a single trusted issuer. Moreover, their work also supports credential delegation in a decentralized setting. Jia et al. [8] proposed a decentralized credential management system with built-in supervision capability. In this system, more than a threshold number of supervisors could trace the identity of a credential user, enabling verifiers to link the credentials of malicious users. This design ensured that the credential system remained robust against misbehaving users. Sun et al. [9] proposed a multi-authority credential system that integrates credential delegation with revocation and traceability. In this system, users can obtain aggregatable credentials from multiple issuers and selectively disclose attributes, while designated authorities are able to revoke invalid or expired credentials and trace the identities of malicious users. Sonnino et al. [10] proposed Coconut, a threshold-issued selective disclosure credential scheme. When deployed on a blockchain, it ensured the confidentiality and authenticity of credentials even when some issuing authorities were malicious or offline. Built upon Coconut [10], Shi et al. [11] retain the core properties of Coconut while further enabling threshold tracing of user identities and credential revocation, thereby achieving accountability against malicious users.

However, the above schemes by default consider issuer identities to be public, which may leak information beyond the credential content and thus motivate research on issuer-hiding credentials.

B. Issuer-Hiding Credentials

Bobolz et al. [12] first proposed an issuer-hiding credential system, in which the verifier specifies a set of acceptable issuers in advance, and the users can prove that their credentials were issued by one of these issuers without revealing the specific issuer. Following the idea of Bobolz et al. [12], Sanders et al. [13] optimized the Pointcheval-Sanders signature [14] and hid the issuer within a set of candidate issuers without relying on zero-knowledge proofs, thereby improving the efficiency of credential construction and verification. Shi et al. [15] proposed a double issuer-hiding attribute-based credential scheme. In this scheme, a central authority endorses public keys for a group of issuers, and subsequent credential issuance and verification do not reveal the identity of the issuer. The authors provided a concrete instantiation of the credential system using structure-preserving signatures on equivalence classes and tag-based aggregatable mercurial signatures. Mir et al. [5] extended issuer hiding to multi-authority credential systems, enabling both jointly issued credentials and the privacy of issuer identities. They proposed an anonymous credential system for decentralized identity that simultaneously satisfies both the multi-authority and issuer-hiding properties. Specifically, they constructed aggregate signatures with randomizable tags and public keys, as well as aggregate mercurial signatures, and employed these two novel signature primitives to build credential systems. Yu et al. [16] employed many-to-one signatures to construct a multi-issuer credential system with issuer-hiding support. In their scheme, issuer accountability is limited to a single tracing operation and does not allow identifying all issuing authorities that participated in the issuance of a credential.

However, when multiple issuers jointly issue credentials, existing issuer-hiding approaches are unsuitable for applications requiring accountability for malicious issuers. For decentralized systems such as Web 3.0, it is necessary to design more suitable credential schemes that balance both multi-issuer hiding and their accountability.

III. PRELIMINARIES

This section introduces the preliminaries, including bilinear pairing, Bilinear Diffie-Hellman assumption, proof system, and notations used throughout the paper.

A. Notations

Let $\mathcal{S}$, $\mathcal{V}$, and $\mathcal{C}$ denote the set of all signers, the set of all verifiers, and the subset of signers participating in a specific signature, respectively. Let $\mathcal{I}$ and $\mathcal{P}$ respectively represent the set of all issuers and the set of all service providers in the credential system. Let $\mathcal{N} = \mathcal{S} \cup \mathcal{V}$, $\mathcal{T} = \mathcal{I} \cup \mathcal{P}$. Let $|\mathcal{S}|$ be the size of set $\mathcal{S}$. For ease of construction and exposition, we encode the elements in $\mathcal{S}$, $\mathcal{V}$, $\mathcal{I}$ and $\mathcal{P}$ as group elements.

B. Bilinear Pairing

A bilinear mapping is an algebraic operation defined over an elliptic curve and is commonly used in pairing-based cryptography. We adopt Type-II bilinear pairing [17]. Let $\mathbf{G}, \mathbf{G}_T$ be

two cyclic groups of order p . The bilinear mapping is defined as $e : \mathbf{G} \times \mathbf{G} \rightarrow \mathbf{G}_T$, and satisfies the following three properties:

- *Bilinearity*: For all $X, Y \in \mathbf{G}$, and all $\hat{a}, \hat{b} \in \mathbb{Z}_p$, it holds $e(X^{\hat{a}}, Y^{\hat{b}}) = e(X, Y)^{\hat{a}\hat{b}}$.
- *Non-degeneracy*: It holds $e(g, g) \neq 1_{\mathbf{G}_T}$, where g is the generator of $\mathbf{G}$, $1_{\mathbf{G}_T}$ is the identity element of $\mathbf{G}_T$.
- *Efficient Computability*: For any $X, Y \in \mathbf{G}$, computing $e(X, Y)$ is efficient.

C. Bilinear Diffie-Hellman Assumption

The security of pairing-based cryptography is typically guaranteed by computational hardness assumptions. Among them, Bilinear Diffie-Hellman (BDH) assumption [18] is a commonly used in multi-signature schemes. Let $\text{Adv}_{\mathcal{A}}^{\text{BDH}}(1^\lambda)$ denote the advantage of $\mathcal{A}$ in solving the BDH problem. (1) holds.

$$\begin{aligned} \text{Adv}_{\mathcal{A}}^{\text{BDH}}(1^\lambda) &= P[\mathcal{A}(P, P^\alpha, P^\beta, P^\gamma) = e(P, P)^{\alpha\beta\gamma}] \\ &\leq \text{negl}(1^\lambda) \end{aligned} \quad (1)$$

D. Proof System

A proof system [19] for relation $R_\lambda = \{R \subseteq S \times W\}$ consists of a pair of interactive machines (P, V) . The prover $P(s, w)$ takes a statement $s \in S$ and a witness $w \in W$ as input, while the verifier $V(s)$ receives the statement s . After interaction, the verifier outputs 1 if it accepts the statement and 0 otherwise. The proof system satisfies the following properties:

- *Perfect completeness*: The proof system (P, V) is perfect completeness, if for all $(s, w) \in R$, it has $P[P(s, w); V(s) = 1] = 1$.
- *Honest verifier zero knowledge*: The proof system (P, V) is Honest Verifier Zero Knowledge (HVZK) if there exists a simulator Sim such that, for any $(s, w) \in R$, $\text{Sim}(s)$ is computationally indistinguishable from the transcript $\{P(s, w); V(s)\}$. Let $\text{Adv}_{\mathcal{A}(P, V)}^{\text{HVZK}}(1^\lambda)$ denote the distinguishing advantage of an adversary $\mathcal{A}$ in distinguishing between $\text{Sim}(s)$ and $\{P(s, w); V(s)\}$. For any PPT adversary, $\text{Adv}_{\mathcal{A}(P, V)}^{\text{HVZK}}(1^\lambda)$ is negligible.

E. ElGamal Encryption

ElGamal encryption [20] is a public-key encryption scheme consisting of key generation, encryption, and decryption algorithms. It is considered semantically secure if any polynomial-time adversary has only negligible advantage in distinguishing the ciphertexts corresponding to two plaintexts.

IV. PRIVATELY ACCOUNTABLE AND PRIVATELY VERIFIABLE MULTI-SIGNATURE

This section provides the detailed design of PAPVMS and presents the corresponding correctness and security proofs, followed by a comparative analysis with existing protocols.

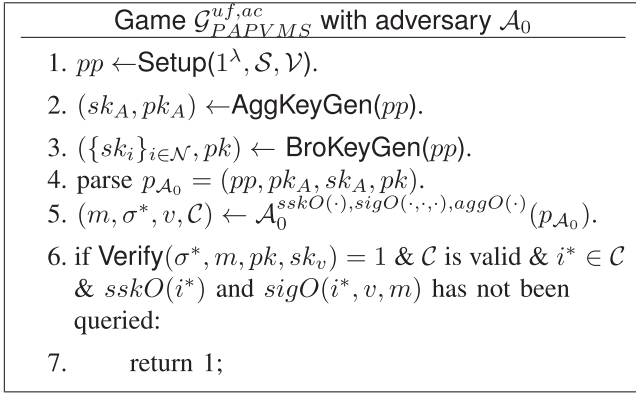


Fig. 1. The PAPVMS unforgeability and accountability game.

A. Formal Definition

PAPVMS consists of eight algorithms: Setup(), AggKeyGen(), GroSec(), BroKeyGen(), Sig(), SignAgg(), Verify(), and Trace(). They are defined as follows.

- $\text{Setup}(1^\lambda, \mathcal{S}, \mathcal{V}) \rightarrow pp$: The algorithm takes as input the security parameter 1^λ , the signer set $\mathcal{S}$, and the verifier set $\mathcal{V}$, outputs the public parameter pp .
- $\text{AggKeyGen}(pp) \rightarrow (sk_A, pk_A)$: The algorithm takes the public parameter pp as input and outputs the key pair (sk_A, pk_A) for the regulator.
- $\text{GroSec}(pp) \rightarrow x_i$: The algorithm takes the public parameter pp as input and outputs the shared random number x_i .
- $\text{BroKeyGen}(pp) \rightarrow (pk, \{sk_i\}_{i \in \mathcal{N}})$: The algorithm takes the public parameter pp as input and outputs the verification key pk and the secret key sk_i of i .
- $\text{Sig}(sk_i, v, m) \rightarrow \hat{\sigma}_i$: The algorithm takes as input the secret key sk_i of signer i , the verifier identity v , and the message m , and outputs the corresponding signature $\hat{\sigma}_i$.
- $\text{SignAgg}(\{\hat{\sigma}_i\}_{i \in \mathcal{C}}, pk_A) \rightarrow (\sigma, C_\sigma)$: The algorithm takes as input the signatures of the set $\mathcal{C}$, where $\mathcal{C}$ denotes the set of signers participating in the current signing process, together with pk_A , and outputs the aggregated signature σ together with the ciphertext C_σ obtained by encrypting $\mathcal{C}$.
- $\text{Verify}(\sigma, m, pk, sk_v) \rightarrow \{0/1\}$: The algorithm takes as input the aggregated signature σ , the message m , the verification key pk , and the secret key sk_v of the verifier. It outputs 1 if the signature is accepted and 0 otherwise.
- $\text{Trace}(sk_A, C_\sigma) \rightarrow \mathcal{C}$: The algorithm takes as input the ciphertext C_σ and the tracing key sk_A , and outputs the set $\mathcal{C}$.

B. Security Model

Let the adversary $\mathcal{A}_0$ participate in the unforgeability and accountability game $\mathcal{G}_{PAPVMS}^{uf,ac}$ as defined in Fig. 1. The adversary $\mathcal{A}_0$ is granted access to three types of oracles, as illustrated in Fig. 2, namely the $sskO()$, $sigO()$, and the $aggO()$. Specifically, $sskO()$ outputs the secret key of a signer or a verifier queried by the adversary, $sigO()$ returns the signature generated by a

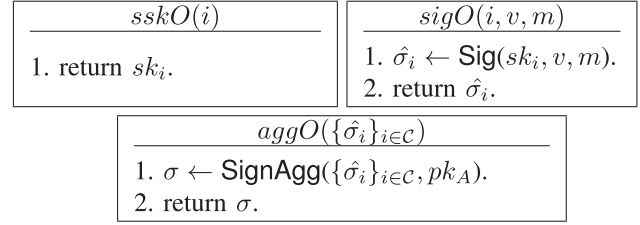
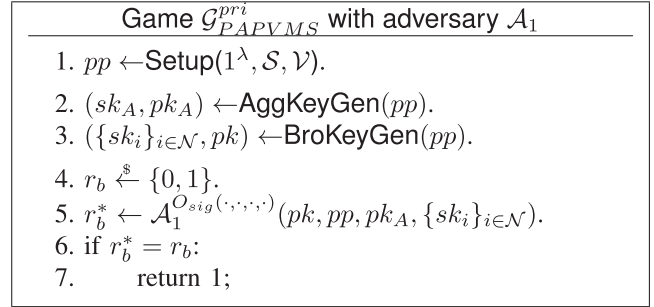
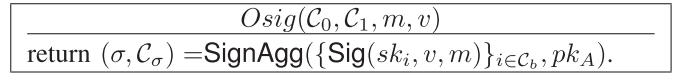
Fig. 2. Oracles for $\mathcal{A}_0$.

Fig. 3. The PAPVMS privacy game.

Fig. 4. Oracles for $\mathcal{A}_1$.

queried signer on a specified message, $aggO()$ returns to the aggregator the aggregated result of the signature fragments, i.e., the final signature. The adversary $\mathcal{A}_0$ wins the game if it produces a valid signature that is attributable to $\mathcal{C}$, among which $|\mathcal{C} - 1|$ signers are corrupted and at least one signer is honest. The game $\mathcal{G}_{PAPVMS}^{uf,ac}$ is illustrated in Fig. 1. Let the advantage of $\mathcal{A}_0$ in winning $\mathcal{G}_{PAPVMS}^{uf,ac}$ be $\text{Adv}_{\mathcal{A}_0}^{\mathcal{G}_{PAPVMS}^{uf,ac}}(1^\lambda)$.

Fig. 3 illustrates the privacy game $\mathcal{G}_{PAPVMS}^{pri}$ for PAVAMS. The PPT adversary $\mathcal{A}_1$ is granted access to the oracle $O_{sig}()$ shown in Fig. 4. The oracle takes as input two signer sets $\mathcal{C}_0$ and $\mathcal{C}_1$, along with a message m , and returns a signature on m generated by one of the two signer sets. If such an adversary cannot output the correct signer set with non-negligible advantage, then the signers in PAVAMS are considered private. Let the advantage of $\mathcal{A}_1$ in winning $\mathcal{G}_{PAPVMS}^{pri}$ be $\text{Adv}_{\mathcal{A}_1}^{\mathcal{G}_{PAPVMS}^{pri}}(1^\lambda)$.

C. PAPVMS Construction

The construction of PAPVMS is shown in Fig. 5.

In the initialization phase, the system executes the $\text{Setup}(1^\lambda, \mathcal{S}, \mathcal{V})$. It generates the public parameter pp , which includes three hash functions H_V, H_K, H_M , two bilinear groups $\mathbf{G}, \mathbf{G}_T$ of prime order p , and the bilinear map $e : \mathbf{G} \times \mathbf{G} \rightarrow \mathbf{G}_T$. Where g denotes the generator of the bilinear group $\mathbf{G}$, $\mathcal{M}$ denotes the message space.

To obtain its own key pair, the regulator invokes $\text{AggKeyGen}(pp)$ to obtain (sk_A, pk_A) . Each verifier v_i first invokes

Privately Accountable and Privately Verifiable Multi-Signature

- **Setup**($1^\lambda, \mathcal{S}, \mathcal{V}$):
 1. Generate $(\mathbf{G}, \mathbf{G}_T, e, p, g)$.
 2. Set $H_V: \mathbf{G} \rightarrow \mathbb{Z}_p$, $H_K: \mathcal{S}, \mathcal{V} \rightarrow \mathbf{G}$, and $H_M: \mathcal{M} \rightarrow \mathbf{G}$.
 3. Return $pp \leftarrow (\mathbf{G}, \mathbf{G}_T, e, p, g, H_V, H_K, H_M, \mathcal{S}, \mathcal{V})$.
- **AggKeyGen**(pp):
 1. Choose a random number $sk_A \in \mathbb{Z}_p$ as the private key.
 2. Compute and broadcast $pk_A = g^{sk_A}$.
 3. Return (sk_A, pk_A) .
- **GroSec**(pp):
 1. If $|v_j| = 1$, return a random number $x_j \leftarrow \mathbb{Z}_p$. Else:
 2. Choose a random number $\gamma_i \in \mathbb{Z}_p$. // where i is a member in v_j
 3. Compute $\rho_i = g^{\gamma_i}$ and broadcast it to the other members in v_j .
 4. Parse the received values as $\{\rho_1, \dots, \rho_q\}$. // where $|v_j| = q$
 5. For $i \in v_j$, compute $\varrho_i = (\frac{\rho_{i+1}}{\rho_{i-1}})^{\gamma_i}$ and $\varphi_i = \rho_{i-1}^{q \cdot \gamma_i} \cdot \prod_{y=0}^{q-2} \varrho_{i+y}^{q-1-y} = g^{\sum_{i=1}^q \gamma_i \gamma_{i+1}}$.
 6. Return $x_j = H_V(\varphi_i)$.
- **BroKeyGen**(pp):
 1. For $i \in \mathcal{S}$, choose a random number $x_i \in \mathbb{Z}_p$.
 2. For $i \in \mathcal{N} \setminus \{j\}$, compute $H_K(j)^{x_i}$ and g^{x_i} , then broadcast them.
 3. Return $pk = \prod_{j \in \mathcal{N}} g^{x_j}$ and $sk_i = \prod_{j \in \mathcal{N}} H_K(i)^{x_j}$.
- **Sig**(sk_i, v, m):
 1. Choose a random number $r_i \in \mathbb{Z}_p$.
 2. Compute $\sigma_{0,i} = g^{r_i} \cdot H_K(v)^{r_i}$, $\sigma_{1,i} = sk_i \cdot H_M(m)^{r_i}$.
 3. Return $\hat{\sigma}_i = (\sigma_{0,i}, \sigma_{1,i})$.
- **SignAgg**($\{\hat{\sigma}_i\}_{i \in \mathcal{C}}, pk_A$):
 1. Choose a random number $r_\sigma, r_c \in \mathbb{Z}_p$.
 2. For $i \in \mathcal{C}$, parse $\hat{\sigma}_i = (\sigma_{0,i}, \sigma_{1,i})$.
 3. Compute $\sigma_0 \leftarrow (\prod_{i \in \mathcal{C}} \sigma_{0,i})^{r_\sigma}$, $\sigma_1 \leftarrow (\prod_{i \in \mathcal{C}} \sigma_{1,i})^{r_\sigma}$, $\sigma_2 = (\prod_{i \in \mathcal{C}} H_K(i))^{r_\sigma}$, and a proof π_σ for:

$$R(\sigma_2, k; \{H_K(i)\}_{i \in \mathcal{C}}, r_\sigma) = \text{true if } \sigma_2 = (\prod_{i \in \mathcal{C}} H_K(i))^{r_\sigma} \wedge k = |\mathcal{C}|.$$
 4. Compute $c_0^\sigma = g^{r_c}$, $c_1^\sigma = \mathcal{C} \cdot pk_A^{r_c}$, $c_2^\sigma = g^{r_\sigma} \cdot pk_A^{r_c}$.
 5. Return $\sigma = (\sigma_0, \sigma_1, \sigma_2)$, $\mathcal{C}_\sigma = (c_0^\sigma, c_1^\sigma, c_2^\sigma, \pi_\sigma)$.
- **Verify**(σ, m, pk, sk_v):
 1. Parse σ as $\sigma = (\sigma_0, \sigma_1, \sigma_2)$.
 2. If $e(\sigma_1, g \cdot H_K(v)) = e(\sigma_2, pk \cdot sk_v) \cdot e(H_M(m), \sigma_0)$ hold, return 1.
- **Trace**($sk_A, \mathcal{C}_\sigma$):
 1. Parse $\mathcal{C}_\sigma$ as (c_0^σ, c_1^σ) .
 2. Compute $s = (c_0^\sigma)^{sk_A}$, and $\mathcal{C} = c_1^\sigma \cdot s^{-1}$.
 3. Return $\mathcal{C}$.

Fig. 5. Privately Accountable and Privately Verifiable Multi-Signature.

GroSec(pp) to generate a random value x_i . If a verifier j chooses not to share its key, then it independently holds the corresponding random value x_j and sets $|v_j| = 1$, where $|v_j|$ denotes the number of parties jointly holding the private key of verifier j . If multiple verifiers share the same private key, they are associated with the same identity j and the same value x_j . In this case, the size of v_j satisfies $|v_j| > 1$. Subsequently, both signers and verifiers invoke BroKeyGen(pp) to obtain the verification key pk and their respective secret keys $\{sk_i\}_{i \in \mathcal{N}}$.

In the signing phase, each signer i first calls Sig(sk_i), v , m to generate a signature $\hat{\sigma}_i$, where v is the designed verifier. Then, the aggregator collects all signatures, defines the set of all participating signers as $\mathcal{C}$, and calls SignAgg($\{\hat{\sigma}_i\}_{i \in \mathcal{C}}, pk_A$) to produce the aggregated signature σ , the ciphertext $\mathcal{C}_\sigma$ of $\mathcal{C}$, and a proof π_σ . π_σ can be used to prove that the signature is correctly aggregated and that $|\mathcal{C}|$ is equal to k . During aggregation, the aggregator inevitably learns the identities of the signers. Therefore, assigning the aggregator the responsibility for signer

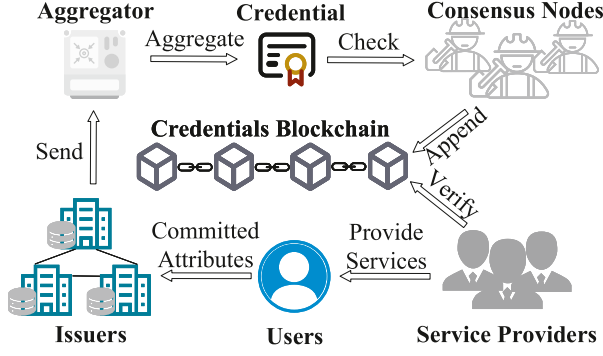


Fig. 6. PAPV-MIC system model.

identity hiding and accountability ensures minimal disclosure of signer identities.

In the verification phase, the verifier calls $\text{Verify}(\sigma, m, pk, sk_v)$. Since the verifier's secret key sk_v is included as part of the input, only the verifier possessing the secret key can verify the signature, thereby achieving the designated verifier property. In this process, the verifier only learns that the signers belong to a set trusted by the verifier and the number of participating signers, while the individual identities of the signers remain hidden. When it is necessary to disclose the signers involved in a particular signature, the regulator calls algorithm $\text{Trace}(sk_A, C_\sigma)$ to reveal the set of signers $\mathcal{C}$.

When PAPVMS is accountable rather than privately accountable, Step 2 of $\text{SignAgg}(\{\hat{\sigma}_i\}_{i \in \mathcal{C}}, pk_A)$ is modified to $\sigma_0 \leftarrow \prod_{i \in \mathcal{C}} \sigma_{0,i}$, $\sigma_1 \leftarrow \prod_{i \in \mathcal{C}} \sigma_{1,i}$. The signature σ_2 is replaced with $\prod_{i \in \mathcal{C}} H_{\mathcal{K}}(i)$. In this case, the resulting signature takes the form $\sigma = (\sigma_0, \sigma_1, \mathcal{C})$. At this point, to enable the computation of $\prod_{i \in \mathcal{C}} H_{\mathcal{K}}(i)$, we modify algorithm $\text{Verify}(\sigma, m, pk, sk_v)$ into $\text{Verify}(\sigma, m, pk, sk_v, \mathcal{C})$ which incorporates the signer set $\mathcal{C}$.

D. Correctness

(2) presents the correctness proof of PAPVMS verification. This verification requires the verifier's secret key sk_v . Therefore, the signature verification remains private. The $\text{Trace}()$ algorithm requires the tracing key sk_A . Therefore, accountability is also private. If $e(\sigma_2, g) = e(\prod_{i \in \mathcal{C}} H_{\mathcal{K}}(i), g^{r_\sigma})$ holds, then the $\mathcal{C}$ output by the execution of $\text{Trace}()$ is considered correct, where g^{r_σ} can be obtained by c_0^g and c_2^g via decryption. The correctness proof is given in (3).

$$\begin{aligned}
 e(\sigma_1, g \cdot H_{\mathcal{K}}(v)) &= e\left(\left(\prod_{i \in \mathcal{C}} sk_i \cdot H_{\mathcal{M}}(m)^{r_i}\right)^{r_\sigma}, g \cdot H_{\mathcal{K}}(v)\right) \\
 &= e\left(\prod_{i \in \mathcal{C}} H_{\mathcal{K}}(i)^{r_\sigma}, pk \cdot sk_v\right) \\
 &\cdot e\left(H_{\mathcal{M}}(m)^{r_\sigma}, \left(\prod_{i \in \mathcal{C}} g \cdot H_{\mathcal{K}}(v)\right)^{r_i}\right) \\
 &= e(\sigma_2, pk \cdot sk_v) \cdot e(H_{\mathcal{M}}(m), \sigma_0)
 \end{aligned} \tag{2}$$

$$e(\sigma_2, g) = e\left(\left(\prod_{i \in \mathcal{C}} H_{\mathcal{K}}(i)\right)^{r_\sigma}, g\right) = e\left(\left(\prod_{i \in \mathcal{C}} H_{\mathcal{K}}(i)\right), g^{r_\sigma}\right) \tag{3}$$

E. Unforgeability and Accountability Proof

The unforgeability and accountability of PAPVMS is demonstrated through the interaction between adversary $\mathcal{A}_0$, and the simulator $\mathcal{A}_0^*$. Specifically, adversary $\mathcal{A}_0$ engages in the $\mathcal{G}_{PAPVMS}^{uf,ac}$ game through queries interactions, while $\mathcal{A}_0^*$ solves the BDH problem based on the value returned by $\mathcal{A}_0$.

$\mathcal{A}_0^*$ is given a tuple $(G, G_T, g^\alpha, g^\beta, g^\gamma, e, p, g)$ and sends it to adversary $\mathcal{A}_0$. Subsequently, $\mathcal{A}_0^*$ forwards (G, G_T, e, p, g) as the public parameter to adversary $\mathcal{A}_0$. Then $\mathcal{A}_0$ designates the signer set $\mathcal{S}$ and the verifier set $\mathcal{V}$. Before $\mathcal{A}_0$ issues any queries, $\mathcal{A}_0^*$ performs the following initialization steps.

- Designate, but do not reveal, the identity $s^* \in \mathcal{S}$ of the unique honest signer.
- Designate, but do not reveal, a message $\mu \in \mathcal{M}$.
- Randomly choose $(x_1, \dots, x_{s^*-1}, x_{s^*+1}, \dots, x_{|\mathcal{S}|})$ from $\mathbb{Z}_p$.
- Randomly choose $(y_1, \dots, y_{|\mathcal{V}|})$ from $\mathbb{Z}_p$.
- Compute $pk = g^\alpha$, and send pk to $\mathcal{A}_0$.

Subsequently, $\mathcal{A}_0$ issues a series of queries to $\mathcal{A}_0^*$. The corresponding queries and responses are described as follows.

- $H_{\mathcal{K}}(i)$:
 - if $i \neq s^*$ and $i \in \mathcal{S}$: return g^{x_i} and record g^{x_i} ,
 - if $i \neq s^*$ and $i \in \mathcal{V}$: return g^{y_i} and record g^{y_i} ,
 - else if $i = s^*$: return g^β .

- $H_{\mathcal{M}}(m)$:
 - if $m = \mu$, $\mathcal{A}_0^*$ aborts.
 - else, $\mathcal{A}_0^*$ uses a random coin ω_m that takes the value 1 with probability $\frac{1}{q_{sig}}$, and 0 otherwise, where q_{sig} denotes the number of times $\mathcal{A}_0$ queries $sigO(\cdot, \cdot, \cdot)$, then:

case 1: $\omega_m = 0$, $\mathcal{A}_0^*$ randomly selects $r_m \in \mathbb{Z}_p$, returns $g^{r_m + \alpha \cdot (m - \mu)}$.

case 2: $\omega_m = 1$, $\mathcal{A}_0^*$ randomly selects $r_m^* \in \mathbb{Z}_p$, returns $g^{r_m^*}$ and record $g^{r_m^*}$.

- $sskO(i)$:
 - if $i \in \mathcal{S}$:
 - case 1: $i \neq s^*$, return $g^{\alpha \cdot x_i}$.
 - case 2: $i = s^*$, $\mathcal{A}_0^*$ aborts.
 - else if $i \in \mathcal{V}$: return $g^{\alpha \cdot y_i}$ and record $g^{\alpha \cdot y_i}$.

- $sigO(i, v, m)$:
 - if $i \neq s^*$, randomly select $r \in \mathbb{Z}_p$, query oracle $H_{\mathcal{M}}(m)$, and return $(g^{y_v \cdot r + r}, g^{\alpha \cdot x_i} \cdot H_{\mathcal{M}}(m)^r)$ as $\hat{\sigma}_i$.
 - if $i = s^*$ and $\omega_m = 0$, return $(g^{(r + \frac{-\beta}{m - \mu})} \cdot y_v + (r + \frac{-\beta}{m - \mu}), H_{\mathcal{M}}(m)^r \cdot g^{\frac{-r \cdot \beta}{m - \mu}})$ as $(\sigma_{0,s^*}, \sigma_{1,s^*})$.

When $i \neq s^*$, the $sskO(\cdot)$ response is $g^{\alpha \cdot x_i} = (g^{x_i})^\alpha = H_{\mathcal{K}}(i)^\alpha = sk_i$. When $i \neq s^*$, the $sigO(\cdot, \cdot, \cdot)$ response is $(g^{y_v \cdot r + r}, g^{\alpha \cdot x_i} \cdot H_{\mathcal{M}}(m)^r)$, where $g^{y_v \cdot r + r} = H_{\mathcal{K}}(v)^r \cdot g^r = \sigma_{0,i}$, $g^{\alpha \cdot x_i} \cdot H_{\mathcal{M}}(m)^r = sk_i \cdot H_{\mathcal{M}}(m)^r = \sigma_{1,i}$. When $i = s^* \wedge \omega_m = 0$, the $sigO(\cdot, \cdot, \cdot)$ response is $(g^{(r + \frac{-\beta}{m - \mu})} \cdot y_v + (r + \frac{-\beta}{m - \mu}), H_{\mathcal{M}}(m)^r \cdot g^{\frac{-r \cdot \beta}{m - \mu}})$, let $r' = r - \frac{\beta}{m - \mu}$, since r, r' still retain its randomness. Then

$g^{(r+\frac{-\beta}{m-\mu}) \cdot y_v + (r+\frac{-\beta}{m-\mu})} = (g^{y_v})^{r'} \cdot g^{r'} = H_K(v)^{r'} \cdot g^{r'} = \sigma_{0,s^*}$,
 $H_M(m)^r \cdot g^{\frac{-r\beta}{m-\mu}} = g^{r_m \cdot (r' + \frac{\beta}{m-\mu})} \cdot g^{\alpha \cdot (m-\mu) \cdot (r' + \frac{\beta}{m-\mu})}$.
 $g^{\frac{-r\beta}{m-\mu}} = (g^{r_m})^{r'} \cdot (g^\alpha)^{r' \cdot (m-\mu)} \cdot g^{\alpha \cdot \beta} = (g^{r_m + \alpha \cdot (m-\mu)})^{r'}$.
 $sk_{s^*} = H_M(m)^{r'} \cdot sk_{s^*} = \sigma_{1,s^*}$. Thus, for the above $sskO(\cdot)$ and $sigO(\cdot, \cdot, \cdot)$ queries, the responses received by $\mathcal{A}_0$ are distributed identically to genuine secret keys or signatures.

• $aggO(\{(\sigma_{0,i}, \sigma_{1,i})\}_{i \in \mathcal{C}})$:

1. choose and record a random number $r_\sigma \in \mathbb{Z}_p$.

2. return $\sigma_0 \leftarrow (\prod_{i \in \mathcal{C}} \sigma_{0,i})^{r_\sigma}$, $\sigma_1 \leftarrow (\prod_{i \in \mathcal{C}} \sigma_{1,i})^{r_\sigma}$, $\sigma_2 = (\prod_{i \in \mathcal{C}} H_K(i))^{r_\sigma}$.

Finally, $\mathcal{A}_0$ outputs $(\sigma^*, \mathcal{C}, v, m)$. If $s^* \notin \mathcal{C}$ or $\omega_m = 0$ or $\text{Verify}(\sigma^*, m, pk, sk_v) = 0$ or $\mathcal{C}$ is invalid, $\mathcal{A}_0^*$ aborts. After the above steps, if the game $\mathcal{G}_{PAPVMS}^{uf,ac}$ does not abort, then $\mathcal{A}_0^*$ uses the output of $\mathcal{A}_0$ to solve the BDH problem, as shown in (4)–(7). (4) represents the signature verification. Replacing the symbolic variables in this equation with the concrete oracle outputs yields (5). By rearranging the exponent on the left-hand side, we derive (6). (7) is then obtained by raising both sides of the equation to the same exponent.

It can be seen that the probability that the game does not abort equals $\mathcal{A}_0^*$'s advantage in solving BDH. Let Q_1 denote the event that the queries to $H_M(\cdot)$ do not abort, Q_2 denote the event that the queries to $sigO(\cdot, \cdot, \cdot)$ do not abort, Q_3 denote the event that the designated s^* is the honest signer in σ^* , Q_4 denote the event that $\omega_m = 1$, Q_5 denote the event that $\text{Verify}(\sigma^*, m, pk, sk_v) = 1$ or $\mathcal{C}$ is invalid. $\mathcal{A}_0^*$'s advantage in solving the BDH problem is given by (8), where t_H denotes the number of queries that $\mathcal{A}_0$ makes to $H_M(\cdot)$, and t_S denotes the number of queries that $\mathcal{A}_0$ makes to $sigO(\cdot, \cdot, \cdot)$.

$$e\left(\prod_{i \in \mathcal{C}} H_K(i)^{r_\sigma^*}, pk \cdot sk_v\right) = \frac{e(\sigma_1, g \cdot H_K(v))}{e(\mathcal{M}(m), \sigma_0)} \quad (4)$$

$$e\left(g^{\beta \cdot r_\sigma^*}, g^{\alpha(1+y_v)}\right) = \frac{e(\sigma_1, g \cdot H_K(v))}{e(g^{r_m^*}, \sigma_0) \cdot e(\prod_{i \in \mathcal{C}} g^{x_i \cdot r_\sigma^*}, g^{\alpha(1+y_v)})} \quad (5)$$

$$e(g, g)^{\alpha \cdot \beta} = \frac{e(\sigma_1, g^{r_\sigma^{*-1}})}{e(\prod_{i \in \mathcal{C}} g^{x_i \cdot r_\sigma^*}, g^\alpha) \cdot e(g^{r_m^* \cdot (1+y_v)^{-1} \cdot r_\sigma^*}, \sigma_0)} \quad (6)$$

$$e(g, g)^{\alpha \cdot \beta \cdot \gamma} = \frac{e(\sigma_1, g^{\gamma \cdot r_\sigma^{*-1}})}{e(\prod_{i \in \mathcal{C}} g^{\gamma \cdot x_i \cdot r_\sigma^{*-1}}, g^\alpha) \cdot e(g^{\gamma \cdot r_m^* \cdot (1+y_v)^{-1} \cdot r_\sigma^{*-1}}, \sigma_0)} \quad (7)$$

$$\begin{aligned} \text{Adv}_{\mathcal{A}_0}^{BDH}(1^\lambda) &= P(Q_1 \wedge Q_2 \wedge Q_3 \wedge Q_4 \wedge Q_5) \\ &= P(Q_1) \cdot P(Q_2|Q_1) \cdot P(Q_3|Q_4Q_5) \cdot P(Q_4|Q_1) \cdot P(Q_5) \\ &= \left(\frac{p-1}{p}\right)^{t_H+t_S} \cdot \left(\frac{t_S}{t_S+1}\right)^{t_S} \cdot \frac{1}{|\mathcal{S}|} \cdot \frac{1}{t_S+1} \\ &\quad \cdot \text{Adv}_{\mathcal{A}_0}^{G_{PAPVMS}^{uf,ac}}(1^\lambda) \end{aligned}$$

$$\geq \left(1 - \frac{t_H+t_S}{p}\right) \cdot \frac{1}{e} \cdot \frac{1}{|\mathcal{S}|} \cdot \frac{1}{t_S+1} \cdot \text{Adv}_{\mathcal{A}_0}^{G_{PAPVMS}^{uf,ac}}(1^\lambda) \quad (8)$$

F. Signer Privacy Proof

We prove the signer privacy of PAPVMS through the following game.

Game 0: This is the game shown in Fig. 3. Let E_0 denotes the event that $\mathcal{A}_1$ in $\mathcal{G}_{PAPVMS}^{pri}$ outputs 1, it has (9).

$$\text{Adv}_{\mathcal{A}_1}^{G_{PAPVMS}^{pri}}(1^\lambda) = |2P(E_0) - 1| \quad (9)$$

Game 1: This game is the same as *Game 0*, except that π_σ is replaced by the output of the simulator on input σ_2 . Since the simulator's output is computationally indistinguishable from the real proof, the difference between the adversary's advantage in *Game 0* and that in *Game 1* is negligible. Let $\text{Adv}_{\beta_0(P,V)}^{HVZK}(1^\lambda)$ denote the advantage of an adversary β_0 in distinguishing π_σ from the simulated one, E_1 denote the event that $\mathcal{A}_1$ in *Game 1* outputs 1, it has (10).

$$|P(E_1) - P(E_0)| \leq \text{Adv}_{\beta_0(P,V)}^{HVZK}(1^\lambda) \quad (10)$$

Game 2: This game is the same as *Game 1*, except when responding to σ , it replaces $(\prod_{i \in \mathcal{C}} g^{r_i} H_K(i)^{r_i})^{r_\sigma}$ with $(g^r \cdot H_K(v)^r)^{r_\sigma}$, replaces $(\prod_{i \in \mathcal{C}} sk_i \cdot H_M(m)^{r_i})^{r_\sigma}$ with $(sk_v \cdot H_M(m)^r)^{r_\sigma}$, and replaces $(\prod_{i \in \mathcal{C}} H_K(i))^{r_\sigma}$ with $H_K(v)^{r_\sigma}$, where r is a random number, v is the designated verifier in $\mathcal{V}$. Due to the confidentiality and randomness of r , as well as the validity of the substituted signature, the replacement is computationally indistinguishable to the adversary. Therefore, the difference between the adversary's advantage in *Game 1* and that in *Game 2* is negligible. Let E_2 denote the event that $\mathcal{A}_1$ in *Game 2* outputs 1, it has (11).

$$|P(E_2) - P(E_1)| \leq \frac{1}{p} \quad (11)$$

Game 3: This game is the same as *Game 2*, except when responding to $\mathcal{C}_\sigma$, it replaces $\mathcal{C} \cdot pk_A^{r_c}$ with $v \cdot pk_A^{r_c}$ and replaces $g^{r_\sigma} \cdot pk_A^{r_c}$ with $g^r \cdot pk_A^{r_c}$, where r is a random number. Due to the ciphertext indistinguishability of ElGamal encryption, the replacement is computationally indistinguishable to the adversary. Let $\text{Adv}_{\beta_1}^{ElG}(1^\lambda)$ denote the advantage of β_1 in distinguishing encryptions in the ElGamal encryption scheme. Therefore, the difference between the adversary's advantage in *Game 2* and that in *Game 3* is negligible. Let E_3 denote the event that $\mathcal{A}_1$ in *Game 3* outputs 1, it has (12).

$$|P(E_3) - P(E_2)| \leq \text{Adv}_{\beta_1}^{ElG}(1^\lambda) \quad (12)$$

Finally proof. In *Game 3*, the information obtained by $\mathcal{A}_1$ from $Osig(\cdot, \cdot, \cdot)$ is independent of b . Therefore, it has (13).

$$P(E_3) = \frac{1}{2} \quad (13)$$

By combining Equations (9)–(13), it has (14).

$$\text{Adv}_{\mathcal{A}_1}^{G_{PAPVMS}^{pri}}(1^\lambda) \leq 2(\text{Adv}_{\beta_0(P,V)}^{HVZK}(1^\lambda) + \frac{1}{p} + \text{Adv}_{\beta_1}^{ElG}(1^\lambda)) \quad (14)$$

TABLE I

COMPARISON OF PAPVMS WITH PAIRING-BASED MULTI-SIGNATURE PROTOCOLS. $|\mathbb{G}|$ DENOTES THE SIZE OF GROUP ELEMENTS, n DENOTES THE NUMBER OF SIGNERS, AND $\mathcal{C}$ DENOTES THE SET OF SIGNERS PARTICIPATING IN THE SIGNING PROCESS. THE GROTH ET AL. ROW REFERS TO THE MULTI-SIGNATURE CONSTRUCTED USING GROTH 16 [26], AND THE TRUSTED SETUP COLUMN INDICATES WHETHER THE PROTOCOL RELIES ON A ONE-TIME TRUSTED SETUP.

Protocol	pk size	key aggregation time	aggregation key size	signature aggregation time	signature size	trusted setup	designated Verifier	private accountability
BLS MS [24]	$n \cdot \mathbb{G} $	0	0	$O(\mathcal{C})$	$1 \cdot \mathbb{G} $	×	×	×
BB MS [25]	$n \cdot \mathbb{G} $	0	0	$O(\mathcal{C})$	$2 \cdot \mathbb{G} $	×	×	×
Groth et al. [26], [27]	$7 \cdot \mathbb{G} $	$O(n)$	$\text{poly}(\lambda, n)$	$\text{poly}(\lambda, n)$	$3 \cdot \mathbb{G} $	✓	×	×
Das et al. [21]	$7 \cdot \mathbb{G} $	$O(n^2)$	$n \cdot \mathbb{G} $	$O(n)$	$8 \cdot \mathbb{G} $	✓	×	×
Garg et al. [22]	$7 \cdot \mathbb{G} $	$O(n^2)$	$n \cdot \mathbb{G} $	$O(n)$	$9 \cdot \mathbb{G} $	✓	×	×
Boneh et al. [23]	$1 \cdot \mathbb{G} $	0	0	$O(\mathcal{C})$	$2 \cdot \mathbb{G} $	×	×	×
ASVK-MOSS [16]	$1 \cdot \mathbb{G} $	0	0	$O(\mathcal{C})$	$4 \cdot \mathbb{G} $	×	✓	×
PAPVMS (accountability)	$1 \cdot \mathbb{G} $	0	0	$O(\mathcal{C})$	$2 \cdot \mathbb{G} $	×	✓	×
PAPVMS (private accountability)	$1 \cdot \mathbb{G} $	0	0	$O(\mathcal{C})$	$3 \cdot \mathbb{G} $	×	✓	✓

G. Analysis and Comparison

Table I presents a comparison between PAPVMS and existing multi-signature protocols based on bilinear groups. The comparison mainly includes the computational cost, storage overhead, and functionalities of the protocols. It shows that although the signature sizes of BLS and BB multi-signatures are the same, they require storing the public keys of all signers. In decentralized networks, this requirement increases the interaction frequency between the aggregator (or users) and the blockchain. Especially in blockchain-based scenarios, it also significantly increases the storage and communication overhead. While the schemes in [21], [22] aim to reduce the size of verification key, they still introduce considerable overhead in key aggregation computation. In addition, the signatures produced by these protocols are not short, with sizes of $8 \cdot |\mathbb{G}|$ and $9 \cdot |\mathbb{G}|$, respectively.

Boneh et al. [23] were the first to propose a multi-signature protocol that supports a fixed, minimal-size verification key. Moreover, it does not require key aggregation, and the resulting signatures are very short ($2 \cdot |\mathbb{G}|$). Building upon Boneh's protocol and without compromising its performance efficiency, PAPVMS further introduces private accountability and designated verifier functionalities. When signer accountability is enabled, the signature size of PAPVMS matches that of Boneh's protocol, namely $2 \cdot |\mathbb{G}|$. When private accountability of signers feature is enabled, the protocol incurs only one additional group element, resulting in a total signature size of $3 \cdot |\mathbb{G}|$, which remains highly compact. Compared with ASVK-MOSS [16], PAPVMS produces shorter signatures while supporting private accountability of signers.

V. PRIVATELY ACCOUNTABLE AND PRIVATELY VERIFIABLE MULTI-ISSUER CREDENTIALS

This section presents a privately accountable and privately verifiable multi-issuer credential scheme, built upon PAPVMS. It provides a formal definition of the scheme and detailed construction, followed by the security proofs and experimental analysis.

A. Formal Definition

The formal definition of PAPV-MIC is given below, which consists of five algorithms.

- $\text{CrSetup}(1^\lambda, \mathcal{I}, \mathcal{P}) \rightarrow pp$: The algorithm takes the security parameter 1^λ , the issuer identity set $\mathcal{I}$, and the service provider set $\mathcal{P}$ as input, and outputs the system's public parameters pp .
- $\text{AIPKeyGen}(pp) \rightarrow (sk_A, pk_A, \{sk_i\}_{i \in \mathcal{T}}, pk)$: The algorithm takes the system's public parameters pp as input and outputs the key pair (pk_A, sk_A) , the secret key sk_i of the issuers and the service providers, as well as the verification key pk .
- $\text{Issuance}(\{sk_i\}_{i \in \mathcal{C}}, q, sta, pk_A) \rightarrow cred$: The algorithm takes as input the secret key sk_i of the participating issuer $\mathcal{C}$, the identity q of the designated service provider, the statement sta , and pk_A , and outputs the user credential $cred$.
- $\text{Show}(cred, pk, sk_q) \rightarrow \{0/1\}$: The algorithm takes as input the user credential $cred$, the verification key pk , the secret key sk_q of q . It outputs 1 if the credential is accepted, and 0 otherwise.
- $\text{CreTrace}(cred, sk_A) \rightarrow \mathcal{C}$: The algorithm takes as input the credential $cred$ and the tracing key sk_A , and outputs the issuer set $\mathcal{C}$ of the credential.

B. System Model

As shown in Fig. 6, PAPV-MIC involves six types of entities: service providers, issuers, users, aggregator, consensus nodes, and the credential blockchain. The descriptions of these six entities are as follows.

- **Service Providers**: Service providers act as verifiers of credentials and offer users online services, such as access to protected resources. Before delivering these services, they verify the users' credentials to ensure that the users' permissions or attributes comply with the access control policies defined for the service.
- **Issuers**: Issuers issue credentials to users after verifying their attributes, thereby endorsing the users' attributes or identities. During issuance, multiple issuers can jointly issue a credential for a user.

Game $\mathcal{G}_{MIC}^{uf,ac}$ with adversary $\mathcal{A}_2$
1. $pp \leftarrow \text{CrSetup}(1^\lambda, \mathcal{I}, \mathcal{P})$. 2. $(sk_A, pk_A, \{sk_i\}_{i \in \mathcal{T}}, pk) \leftarrow \text{AIPKeyGen}(pp)$. 3. $(cred^*, \mathcal{C}, q) \leftarrow \mathcal{A}_0^{skgO(\cdot), creO(\cdot, \cdot, \cdot)}(pk_A, sk_A, pk)$. 4. if $\text{Show}(cred^*, pk, sk_q) = 1$ & $\mathcal{C}$ is valid & $cred^*$ is not generate by $creO(\cdot, \cdot, \cdot)$ & not all $i \in \mathcal{C}$ have been queried to $skgO(i)$. 5. return 1;

Fig. 7. The PAPV-MIC unforgeable and accountability game.

$skgO(i)$	$creO(\mathcal{C}, q, sta)$
1. return sk_i.	1. $\text{Issuance}(\{sk_i\}_{i \in \mathcal{C}}, q, sta, pk_A) \rightarrow cred$. 2. return $cred$.

Fig. 8. Oracles for $\mathcal{A}_2$.

- **Users:** Users request credentials from issuers and present these credentials to service providers in privacy-preserving manner for online services.
- **Aggregator:** Aggregators are responsible for collecting partial credentials generated by different issuers and aggregating them into a complete credential for the user. As the entity that needs to be aware of the identities of the issuers, the aggregator is responsible for issuer hiding and is therefore assumed to be semi-honest. By holding the tracing key, the aggregator can also act as a regulator in the system. In this case, as a regulator, the aggregator must be trusted.
- **Consensus Nodes:** Consensus nodes verify the legitimacy of credential issuance, such as whether the aggregator correctly aggregates the partial credentials from all issuers. After verification, the consensus nodes use a designated consensus algorithm to append the user's credential and issuance record to the credential blockchain.
- **Credential Blockchain:** Credential blockchain records users' credentials and their issuance records. By jointly maintaining the credential blockchain, users retain control over their identity data. Moreover, in PAPV-MIC, the verification does not require service providers to aggregate issuers' public keys. Therefore, the credential blockchain does not need to store issuers' public keys or related indexing information.

C. Security Model

Figs. 7 and 8 respectively present the unforgeability and accountability game of PAPV-MIC, as well as the oracles accessible to the adversary. If an adversary $\mathcal{A}_2$ who can access the oracles in Fig. 8 wins in game $\mathcal{G}_{MIC}^{uf,ac}$, the credential in PAPV-MIC is considered successfully forged, and $\mathcal{A}_2$'s advantage in winning is denoted as $\text{Adv}_{\mathcal{A}_2}^{\mathcal{G}_{MIC}^{uf,ac}}(1^\lambda)$.

Game $\mathcal{G}_{MIC}^{pri}$ with adversary $\mathcal{A}_3$
1. $pp \leftarrow \text{CrSetup}(1^\lambda, \mathcal{I}, \mathcal{P})$. 2. $(sk_A, pk_A, \{sk_i\}_{i \in \mathcal{T}}, pk) \leftarrow \text{AIPKeyGen}(pp)$. 3. $r_b \xleftarrow{\\$} \{0, 1\}$. 4. $r_b^* \leftarrow \mathcal{A}_3^{IssO(\cdot, \cdot, \cdot, \cdot)}(pk, pp, \{sk_i\}_{i \in \mathcal{T}}, pk_A)$. 5. if $r_b^* = r_b$: 6. return 1;

Fig. 9. The PAPV-MIC privacy game.

$IssO(\mathcal{C}_0, \mathcal{C}_1, sta, q)$
return $cred = \text{Issuance}(\{sk_i\}_{i \in \mathcal{C}_b}, q, sta, pk_A)$.

Fig. 10. Oracles for $\mathcal{A}_3$.

Figs. 9 and 10 respectively present the issuer privacy game of PAPV-MIC and the oracles accessible to the adversary. Issuer privacy implies that the issuer's identity is hidden, that is, the credential is issuer-hiding. If an adversary $\mathcal{A}_3$ who can access the oracles in Fig. 10 wins in game $\mathcal{G}_{MIC}^{pri}$, the issuer privacy is considered broken, and $\mathcal{A}_3$'s advantage in winning is denoted as $\text{Adv}_{\mathcal{A}_3}^{\mathcal{G}_{MIC}^{pri}}(1^\lambda)$.

D. PAPV-MIC Construction

Fig. 11 presents the detailed construction of PAPV-MIC, which consists of five algorithms. In the initialization, the system invokes $\text{CrSetup}(1^\lambda, \mathcal{I}, \mathcal{P})$ to generate the system parameters pp . In the key generation phase, the issuers, service providers, and the aggregator invoke $\text{AIPKeyGen}(pp)$ to obtain their respective key pairs. In the issuance phase, the issuers $i(i \in \mathcal{C})$ and the aggregator invoke $\text{Issuance}(\{sk_i\}_{i \in \mathcal{C}}, q, sta, pk_A)$ to generate the user's credential $cred$. Service provider invokes $\text{Show}(cred, pk, sk_q)$ to verify the user's credential $cred$. In this process, the service provider learns only the number of participating issuers and the set from which they originate while the identities remain hidden. In the credential accountability, the aggregator invokes $\text{CreTrace}(cred, sk_A)$ to output the set $\mathcal{C}$.

In particular, sk_A can be held by a trusted regulator. In this setting, the aggregator no longer performs $\text{CreTrace}(cred, sk_A)$, nor does it invoke $\text{AIPKeyGen}(pp)$. Instead, the accountability functionality is delegated to a trusted regulator, which invokes $\text{AIPKeyGen}(pp)$ and holds the tracing key sk_A . To minimize the entities that are aware of the issuers' identities, PAPV-MIC lets the aggregator also act as a regulator. Moreover, key splitting or secret sharing techniques can be employed to enable multiple entities to jointly manage sk_A , thereby supporting decentralized tracing. PAPV-MIC achieves a balance between issuer privacy and accountability, ensuring system security while preserving credential privacy. Furthermore, the private verifiability of the generated credentials prevents information leakage during verification, thereby enhancing overall privacy protection.

Privately Accountable and Verifiable Multi-Issuer Credential Scheme

- $\text{CrSetup}(1^\lambda, \mathcal{I}, \mathcal{P})$: System calls $\text{PAPVMS.Setup}(1^\lambda, \mathcal{I}, \mathcal{P})$ and outputs pp .
- $\text{AIPKeyGen}(pp)$:
 1. The aggregator calls $\text{PAPVMS.AggKeyGen}(pp)$ to get the key pair (sk_A, pk_A)
 2. The service provider i calls $\text{PAPVMS.GroSec}(pp)$ to obtain a private value x_i .
//The service providers sharing the private key use the same random value x_i
 3. The issuers and service providers call $\text{PAPVMS.BroKeyGen}(pp)$ to obtain their respective secret keys sk_i as well as the verification key pk .
- $\text{Issuance}(\{sk_i\}_{i \in \mathcal{C}}, q, sta, pk_A)$:
 1. The issuer i in $\mathcal{C}$ calls $\text{PAPVMS.Sig}(sk_i, q, sta)$ to generate $\hat{\sigma}_i$.
 2. The aggregator calls $\text{PAPVMS.SignAgg}(\{\hat{\sigma}_i\}_{i \in \mathcal{C}}, pk_A)$ to generate $cred = (\sigma, \mathcal{C}_\sigma, sta)$.
- $\text{Show}(cred, pk, sk_q)$: The service provider q calls $\text{PAPVMS.Verify}(cred, pk, sk_q)$, and outputs 1 if the credential is successfully verified, and 0 otherwise.
- $\text{CreTrace}(cred, sk_A)$: The aggregator calls $\text{PAPVMS.Trace}(cred, sk_A)$, and outputs the set of credential issuers $\mathcal{C}$.

Fig. 11. Privately accountable and privately verifiable multi-issuer credential scheme.

E. Correctness

In PAPV-MIC, the correctness of credential verification and tracing are guaranteed by the correctness of the underlying signature protocol PAPVMS (as shown in (2) and (3)). Private accountability of PAPV-MIC is guaranteed by the $\text{CreTrace}(\cdot, sk_A)$ algorithm, while private verifiability is ensured by the $\text{Show}(\cdot, \cdot, sk_q, \cdot, \cdot)$ algorithm. That is, accountability of the issuer requires the tracing key sk_A , while verification of the credential requires the secret key sk_q of service provider q . Therefore, under the condition that the secret keys of service provider and tracing key remain secret, both the accountability and verification of the credential are private.

F. Security Proof

The following presents the proofs of credential unforgeability and accountability, as well as issuer identity privacy in PAPV-MIC.

1) *Unforgeability and Accountability Proof*: Let $\mathcal{A}_2$ break the unforgeability and accountability game defined in $\mathcal{G}_{MIC}^{uf,ac}$. $\mathcal{A}_2^*$ participates in $\mathcal{G}_{PAPVMS}^{uf,ac}$.

First, $\mathcal{A}_2 \rightarrow \mathcal{I}, \mathcal{P}$. Simulator calls $\text{CrSetup}(1^\lambda, \mathcal{I}, \mathcal{P})$ to obtain pp , then calls $\text{AIPKeyGen}(pp)$ to obtain (sk_A, pk_A, pk) . Then, $\mathcal{A}_2^*$ designates $i^* \in \mathcal{I}$. When $\mathcal{A}_2^*$ queries the $skgO()$, $creO()$ oracles, $\mathcal{A}_2^*$ generates its responses by querying the $sskO()$, $sigO()$, and $aggO()$ oracles. The corresponding responses are described as follows.

- $skgO(i)$: $\mathcal{A}_2^*$ queries $sskO(i)$, and returns sk_i .
- $creO(\mathcal{C}, q, sta)$: $\mathcal{A}_2^*$ queries $aggO(sigO(\{i\}_{i \in \mathcal{C}}, q, sta))$ to obtain σ , then returns $cred = (\sigma, \mathcal{C}_\sigma, sta)$.

The oracle queries made by $\mathcal{A}_2^*$ in Game $\mathcal{G}_{PAPVMS}^{uf,ac}$ are identically distributed to real signatures. Moreover, the responses received by $\mathcal{A}_2$ to its oracle queries are exactly the same as those received by $\mathcal{A}_2^*$ in Game $\mathcal{G}_{PAPVMS}^{uf,ac}$. Therefore, the queries made by $\mathcal{A}_2^*$ are identically distributed to real credentials.

If $\mathcal{A}_2$'s queries to the $skgO()$, $creO()$ oracles cause any abort in $\mathcal{A}_2^*$'s queries to the $sskO()$, $sigO()$, $aggO()$ oracles, $\mathcal{A}_2^*$ aborts. Finally, $\mathcal{A}_2$ outputs a forged credential $cred^* = (\sigma^*, \mathcal{C}_\sigma^*, sta^*)$ with issuer $\mathcal{C}$, and service provider q . If $\text{Show}(cred^*, pk, sk_q) \neq 1$ or $\mathcal{C}$ is invalid ($\text{CreTrace}(cred^*, sk_A) \neq \mathcal{C}$ or (3) does not hold), $\mathcal{A}_2^*$ aborts. If the game does not abort (denoted by $\overline{\text{Abort}}$), then $\mathcal{A}_2^*$ can use the forged credential $cred^*$ to produce a forged signature σ^* signed by $\mathcal{C}$. Let T_1 denote the event that $\mathcal{A}_2^*$ aborts due to $\text{Show}(cred^*, pk, sk_q) \neq 1$ or $\mathcal{C}$ is invalid. Let T_2 denote the event that $\mathcal{A}_2^*$ aborts due to an abort triggered by a $skgO()$ or $creO()$ oracle query (with probability as described in Section IV-E). it has (15), where q_{cr} denotes the number of queries that $\mathcal{A}_2^*$ makes to $creO()$. The adversary $\mathcal{A}_2^*$ can win only if event $T_1 \wedge T_2$ occurs. Therefore, $\text{Adv}_{\mathcal{A}_2^*}^{\mathcal{G}_{PAPVMS}^{uf,ac}}(1^\lambda) \geq P(T_1 \wedge T_2)$ holds. Conditioned on the event $\overline{T_2}$, if event T_1 occurs, then the adversary $\mathcal{A}_2$ can successfully forge a valid credential. Therefore, we have $P(T_1|T_2) \geq \text{Adv}_{\mathcal{A}_2^*}^{\mathcal{G}_{MIC}^{uf,ac}}(1^\lambda)$.

$$\begin{aligned}
 P(\overline{\text{Abort}}) &= \text{Adv}_{\mathcal{A}_2^*}^{\mathcal{G}_{PAPVMS}^{uf,ac}}(1^\lambda) \geq P(T_1 \wedge T_2) \\
 &= P(T_1|T_2) \cdot P(T_2) \geq \text{Adv}_{\mathcal{A}_2^*}^{\mathcal{G}_{MIC}^{uf,ac}}(1^\lambda) \cdot P(T_2) \\
 &\geq \text{Adv}_{\mathcal{A}_2^*}^{\mathcal{G}_{MIC}^{uf,ac}}(1^\lambda) \cdot \left(\frac{p-1}{p}\right)^{q_{cr}} \cdot \left(\frac{q_{cr}}{q_{cr}+1}\right)^{q_{cr}} \cdot \frac{1}{|\mathcal{I}|} \\
 &\geq \text{Adv}_{\mathcal{A}_2^*}^{\mathcal{G}_{MIC}^{uf,ac}}(1^\lambda) \cdot \left(1 - \frac{q_{cr}}{p}\right) \cdot \frac{1}{e} \cdot \frac{1}{|\mathcal{I}|} \quad (15)
 \end{aligned}$$

$$\text{Adv}_{\mathcal{A}_3}^{\mathcal{G}_{MIC}^{pri}}(1^\lambda) = \text{Adv}_{\mathcal{A}_1}^{\mathcal{G}_{PAPVMS}^{pri}}(1^\lambda) \quad (16)$$

2) *Issuer Privacy Proof*: Let $\mathcal{A}_1$ participate in $\mathcal{G}_{PAPVMS}^{pri}$ and $\mathcal{A}_3$ participate in $\mathcal{G}_{MIC}^{pri}$, where the queries of $\mathcal{A}_3$ to $\text{IssO}(\mathcal{C}_0, \mathcal{C}_1, sta)$ is answered by $\mathcal{A}_1$ in response to queries to $\text{Osig}(\mathcal{C}_0, \mathcal{C}_1, sta)$. If $\mathcal{A}_3$ can win in $\mathcal{G}_{MIC}^{pri}$, then adversary $\mathcal{A}_1$

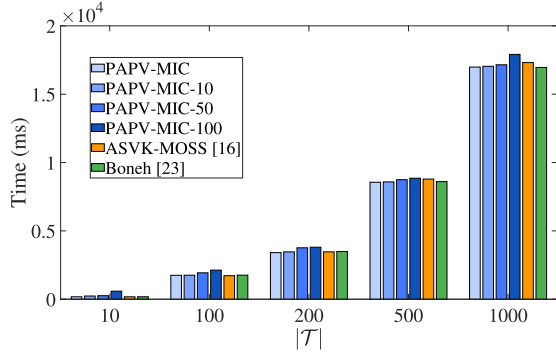


Fig. 12. Time cost of key generation.

can use $\mathcal{A}_3$'s ability to win in $\mathcal{G}_{PAPVMS}^{pri}$. It has (16). Note that issuer privacy implies that the credential is issuer-hiding.

G. Experiments and Analysis

We conduct comparative experiments between PAPV-MIC and the credential systems constructed from Boneh's scheme [23] (abbreviated as Boneh) and ASVK-MOSS [16], since these schemes require neither key aggregation nor the storage of issuer public keys. We compare the time consumption of key generation, credential issuance, presentation, and accountability. Since PAPV-MIC is built upon the underlying primitive PAPVMS, its key management and computational complexity are consistent with those of PAPVMS, as shown in Section IV-G. All experiments are conducted in Java using the JPBC library [28] on a desktop computer equipped with an Intel i7-14700F processor and 32 GB of RAM (31.7 GB available). The bilinear pairing parameters are specified in the *a.properties* file [28].

Fig. 12 illustrates the impact of the total number of issuers and service providers $|\mathcal{T}|$ on the computation cost during the key generation. PAPV-MIC denotes the case with an independent service provider. PAPV-MIC-10, PAPV-MIC-50, and PAPV-MIC-100 correspond to scenarios where 10, 50, and 100 service providers share the secret key, respectively. As shown in Fig. 12, the computation cost of $\text{AIPKeyGen}()$ is positively correlated with the total number of $|\mathcal{T}|$. When all service providers in PAPV-MIC independently hold their own private keys, the execution time of $\text{AIPKeyGen}()$ in PAPV-MIC, ASVK-MOSS, and Boneh's scheme is essentially identical. When multiple service providers need to share a private key, increasing the number of service providers sharing the private key leads to higher computational overhead during the execution of the $\text{AIPKeyGen}()$ algorithm. Specifically, when the total number of issuers and service providers is 1000, a service provider that shares its private key with 100 other service providers incurs a 5.1% higher $\text{AIPKeyGen}()$ computation cost compared with sharing a private key with 10 service providers.

Fig. 13(a) illustrates the relationship between the number of issued credentials and the issuer's computation overhead during the issuance. As illustrated in Fig. 13(a), the issuer's computation cost for executing $\text{Issuance}()$ increases with the number

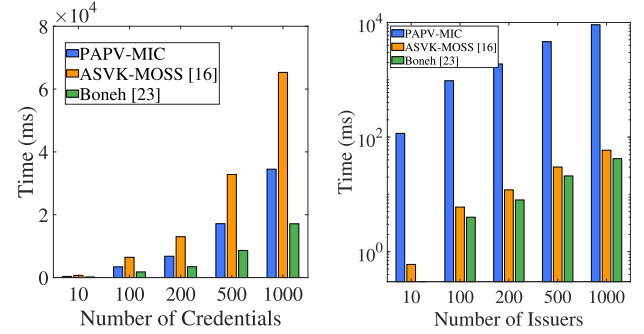


Fig. 13. Time cost of issuance.

of issued credentials. During the issuance, Boneh's scheme incurs the lowest computation cost, followed by PAPV-MIC, while ASVK-MOSS incurs the highest cost. This is because PAPV-MIC shifts part of the issuer's computation to the aggregator, whereas Boneh's scheme achieves the lowest cost since it does not support designated service providers and therefore requires no computation related to service provider identities during issuance. When issuing 1000 credentials, the issuer's computation cost in PAPV-MIC is 89.3% lower than that of ASVK-MOSS and 101.5% higher than that of Boneh's scheme.

Fig. 13(b) illustrates the relationship between the number of issuers for a given credential and the aggregator's computation overhead during the issuance. As shown in Fig. 13(b), the aggregator's running time for executing $\text{Issuance}()$ is positively correlated with the number of issuers. During the issuance, the aggregator under PAPV-MIC incurs the highest computation overhead, followed by ASVK-MOSS, while the Boneh's scheme results in the lowest overhead. The reason is that, in PAPV-MIC, both issuer privacy and accountability are handled by the aggregator. Consequently, the aggregator needs perform additional computations related to issuer identities. When generating a credential jointly issued by 1000 issuers, the aggregator running PAPV-MIC incurs 9.07 seconds more computation time than ASVK-MOSS, and 9.08 seconds more than the aggregator using Boneh's scheme. Nevertheless, in PAPV-MIC, the aggregator is assumed to be equipped with sufficient computing capacity and responsible for issuer privacy and accountability. Therefore, the additional computation overhead introduced for the aggregator in PAPV-MIC remains acceptable.

Fig. 14 illustrates the relationship between the number of credentials and the verification time overhead of the service provider. As shown in Fig. 14, the verification time is positively correlated with the number of credentials. Among the three schemes, the service provider using ASVK-MOSS incurs the lowest verification overhead, followed by PAPV-MIC, while the service provider based on Boneh's scheme exhibits the highest overhead. This is because the credentials generated in ASVK-MOSS are longer than those in PAPV-MIC and Boneh's scheme, which allows the service provider to avoid additional computations during the credential presentation. In contrast, in the Boneh's scheme, issuer identities are public visible.

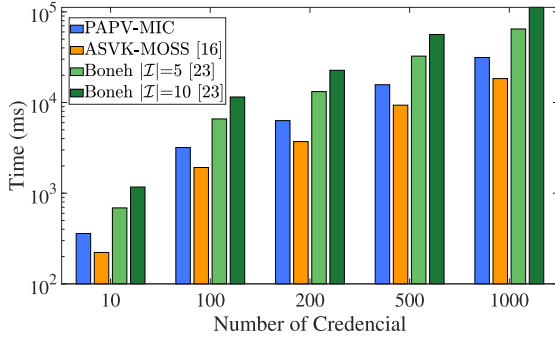


Fig. 14. Time cost of the credential presentation.

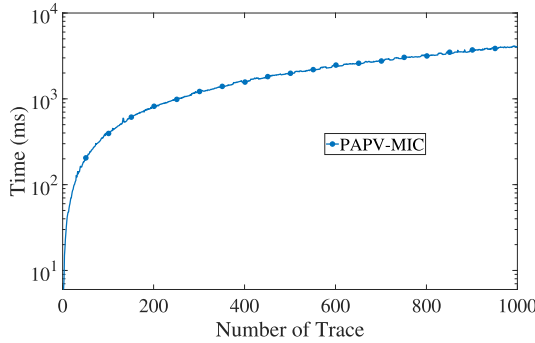


Fig. 15. Time cost of credential accountability.

Therefore, the service provider needs perform issuer-identity-related computations during verification. As a result, both the number of issuers and the number of credentials directly increase the verification overhead under the Boneh's scheme. When verifying 1000 credentials, each issued by 10 issuers, the verification time of the service provider under PAPV-MIC is 70.0% higher than that under ASVK-MOSS, and 72.0% lower than that under the Boneh's scheme.

Fig. 15 illustrates the relationship between the number of accountability queries and the time overhead incurred by the aggregator when performing CreTrace() in PAPV-MIC. In Boneh's scheme, issuer identities are explicitly embedded in the credentials. Therefore, no additional computation is required for accountability. However, their scheme does not support private accountability. In ASVK-MOSS, accountability only determines whether a specific issuer participated in issuing a given credential, rather than revealing the complete set of involved issuers. Therefore, the accountability experiment is conducted solely for PAPV-MIC. From Fig. 15, it can be seen that the accountability time overhead increases with the number of queries and exhibits a logarithmic growth trend, indicating that CreTrace() remains efficient and practically acceptable for large-scale deployments.

The above experimental results indicate that the main computational overhead of PAPV-MIC arises from bilinear pairing operations. The proposed scheme can be directly deployed and efficiently executed on devices with moderate to strong computational capabilities, such as roadside units or high-performance onboard units in vehicular networks. For resource-constrained devices, computationally intensive tasks

can be offloaded through outsourced computation or edge-assisted mechanisms, without leaking sensitive information, thereby reducing the local computational burden and ensuring practical deployability.

VI. CONCLUSION

This paper presents PAPV-MIC, a multi-issuer credential system built on a privately accountable and privately verifiable multi-signature protocol. PAPV-MIC hides issuer identities while preserving accountability, and enforces credential usage control through private verifiability. In addition, it eliminates the need for issuers' public-key aggregation or storage, thereby reducing both computational and storage overhead. As a result, PAPV-MIC provides an accountable and efficient credential solution for decentralized environments especially Web 3.0.

REFERENCES

- [1] Šeila Bećirović Ramić et al., "Selective disclosure in digital credentials: A review," *ICT Exp.*, vol. 10, no. 4, pp. 916–934, 2024.
- [2] T.-A. Ta, X. Hui, and S. C.-K. Chau, "Ring referral: Efficient publicly verifiable ad hoc credential scheme with issuer and strong user anonymity for decentralized identity and more," in *Proc. IEEE Symp. Secur. Privacy*, 2025, pp. 184–202.
- [3] M. Sun, J. Lai, W. Wu, Y. Yang, C.-K. Chu, and R. H. Deng, "How to securely delegate and revoke partial authorization credentials," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 2, pp. 979–996, Mar./Apr. 2025.
- [4] T. Sharma et al., "Future of algorithmic organization: Large-scale analysis of decentralized autonomous organizations (DAOs)," *ACM Trans. Web*, vol. 20, no. 3, pp. 1–25, Feb. 2026.
- [5] O. Mir, B. Bauer, S. Griffy, A. Lysyanskaya, and D. Slamanig, "Aggregate signatures with versatile randomization and issuer-hiding multi-authority anonymous credentials," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 30–44.
- [6] M. Drijvers, S. Gorbunov, G. Neven, and H. Wee, "Pixel: Multi-signatures for consensus," in *Proc. USENIX Secur.*, Aug. 2020, pp. 2093–2110.
- [7] O. Mir, D. Slamanig, and R. Mayrhofer, "Threshold delegatable anonymous credentials with controlled and fine-grained delegation," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 2312–2326, Jul./Aug. 2024.
- [8] M. Jia, J. Chen, Y. Wang, K. He, M. Shi, and R. Du, "Multi-authority anonymous credentials with efficient and decentralized supervision," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 8154–8166, 2025.
- [9] M. Sun et al., "Delegatable multi-authority attribute-based anonymous credentials," *IEEE Trans. Cloud Comput.*, vol. 13, no. 2, pp. 655–666, Apr.–Jun. 2025.
- [10] A. Sonnino, M. Al-Bassam, S. Bano, S. Meiklejohn, and G. Danezis, "Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers," in *Proc. 26th Annu. Netw. Distrib. Syst. Secur. Symp.*, Feb. 2019.
- [11] R. Shi et al., "Threshold attribute-based credentials with redactable signature," *IEEE Trans. Serv. Comput.*, vol. 16, no. 5, pp. 3751–3765, Sep./Oct. 2023.
- [12] J. Bobolz, F. Eidens, S. Krenn, S. Ramacher, and K. Samelin, "Issuer-hiding attribute-based credentials," in *Proc. Cryptol. Netw. Secur.*, 2021, pp. 158–178.
- [13] O. Sanders and J. Traoré, "Compact issuer-hiding authentication, application to anonymous credential," *Proc. Priv. Enhancing Technol.*, vol. 2024, no. 3, pp. 645–658, 2024.
- [14] D. Pointcheval and O. Sanders, "Reassessing security of randomizable signatures," in *Proc. Topics Cryptology. CT-RSA 2018, ser. Lecture Notes Comput. Sci.*, vol. 10808, 2018, pp. 319–338.
- [15] R. Shi et al., "Double issuer-hiding attribute-based credentials from tag-based aggregatable mercurial signatures," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 2585–2602, Jul./Aug. 2024.
- [16] Y. Yu, H. Yang, Y. Li, and X. Du, "Accountable many-to-one signature with short verification key for self-sovereign identity," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 5, pp. 5291–5302, Sep./Oct. 2025.
- [17] S. D. Galbraith, K. G. Paterson, and N. P. Smart, "Pairings for cryptographers," *Discrete Appl. Math.*, vol. 156, no. 16, pp. 3113–3121, 2008.

- [18] K. Liu, C. Wang, and X. Zhou, "Decentralizing access control system for data sharing in smart grid," *High-Confidence Comput.*, vol. 3, no. 2, 2023, Art. no. 100113.
- [19] D. Boneh and C. Komlo, "Threshold signatures with private accountability," in *Proc. Annu. Int. Cryptol. Conf.*, Cham, Switzerland: Springer Nature, 2022, pp. 551–581.
- [20] Y. Zhao et al., "A logarithmic size revocable linkable ring signature for privacy-preserving blockchain transactions," *High-Confidence Comput.*, vol. 5, no. 4, 2025, Art. no. 100319.
- [21] S. Das, P. Camacho, Z. Xiang, J. Nieto, B. Bunz, and L. Ren, "Threshold signatures from inner product argument: Succinct, weighted, and multi-threshold," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 356–370.
- [22] S. Garg, A. Jain, P. Mukherjee, R. Sinha, M. Wang, and Y. Zhang, "hints: Threshold signatures with silent setup," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 3034–3052.
- [23] D. Boneh, A. Partap, and B. Waters, "Accountable multi-signatures with constant size public keys," in *Proc. IACR Int. Conf. Public-Key Cryptogr.*, 2025, pp. 32–65.
- [24] A. Boldyreva, "Threshold signatures, multisignatures and blind signatures based on the gap-diffie-Hellman-group signature scheme," in *Proc. Int. Workshop Public Key Cryptogr.*, 2003, vol. 2567, pp. 31–46.
- [25] S. Lu, R. Ostrovsky, A. Sahai, H. Shacham, and B. Waters, "Sequential aggregate signatures and multisignatures without random oracles," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2006, pp. 465–485.
- [26] J. Groth, "On the size of pairing-based non-interactive arguments," in *Proc. Adv. Cryptology—EUROCRYPT*, 2016, pp. 305–326.
- [27] A. Delignat-Lavaud, C. Fournet, M. Kohlweiss, and B. Parno, "Cinderella: Turning shabby X.509 certificates into elegant anonymous credentials with the magic of verifiable computation," in *Proc. IEEE Symp. Secur. Privacy*, 2016, pp. 235–254.
- [28] A. De Caro and V. Iovino, "jPBC: Java pairing based cryptography," in *Proc. IEEE Symp. Comput. Commun.*, 2011, pp. 850–855.



Haochen Yang (Graduate Student Member, IEEE) received the ME degree in software engineering from Henan University. He is currently working toward the PhD degree with Shaanxi Normal University. His research focuses on security in decentralized networks.



Yong Yu (Fellow, IEEE) is currently a professor with Shaanxi Normal University, China. He has authored more than 100 referred journal articles and conference papers. His research interests include blockchain and cloud security. He also holds the prestigious One Hundred Talent professorship of Shaanxi Province. He is an associate editor for *IEEE Transactions on Dependable and Secure Computing*.



Qian Zhang received the ME degree from Shaanxi Normal University in 2023. She is currently working toward the PhD degree with Xidian University. Her research focuses on blockchain security and applied cryptography.



Keyan Xin received the BE degree in software engineering from Henan University, Henan, China. He is currently working toward the master's degree with Shaanxi Normal University. His research focuses on self-sovereign identity.



Huilin Li received the BS and MS degrees from Shaanxi Normal University in 2018 and 2021, respectively. She is currently working toward the PhD degree with Shaanxi Normal University. Her research focuses on blockchain security and privacy-preserving artificial intelligence.



Dongxiao Yu (Senior Member, IEEE) received the BS degree in mathematics from Shandong University in 2006 and the PhD degree in computer science from The University of Hong Kong in 2014. He is currently a professor with Shandong University. His research interests include wireless networking, distributed computing, and graph algorithms.